

Urban Company- Assignment Report

Date: April 20th, 2025

Author: Vishal Sharma (vishals7654@gmail.com ; 9999898653)



GitHub repository:

https://github.com/VishalSharma0309/uc_no_response/tree/develop

Context

Urban Company is a technology platform that connects customers with various local service professionals. The company operates in multiple cities and offers a range of services, including home maintenance and repairs, beauty and wellness, cleaning, and other household services. Users can use the Urban Company app or website to book services, and the platform connects them with qualified and verified professionals in their area.

Problem Statement

A major pain-point for customers is non-arrival of professionals for their service without prior intimation, (called No-Response) and leads to a bad experience. In this case, accelerated prediction of No-Response enables the platform (Urban Company) to deploy additional strategies to re-assign the booking to another professional.

You need to build a predictive model to predict No-Response probability of a request

Task list

1. Perform a comprehensive EDA
2. Share additional relevant insights and visualizations
3. Build a predictive model to predict no-response given the features described
4. Find the ideal threshold for prediction if
 - a. benefit of a correct no-response prediction is 400,
 - b. benefit of correctly predicting a no-response NOT happening is 0,
 - c. cost of incorrectly predicting a booking as no-response is -200
 - d. cost of incorrectly predicting a booking as NOT no-response is -150
5. Estimate the personalized impact of experiencing an NR on a given request on that customer's LTV

Confusion Matrix

Actual Predicted	False	True
False	TN (0)	FP (-200)
True	FN (-500)	TP (400)

Data Dictionary

Feature Name	Description (based on my understanding)	Type
Unnamed: 0	User ID	Categorical
NR_experienced_on_request	Response (1 means NP)	Binary
no_of_supercats_tried_last_xd	Categories booked in last x days. x can be 30, 90 or 365 days	Numerical
no_of_deliveries_last_xd	Number of deliveries in last x days. x can be 30, 90 or 365 days	Numerical

Feature Name	Description (based on my understanding)	Type
service_deliverd_aov_last_xd	Average order value of services delivered in last x days. x can be 30, 90 or 365 days	Numerical
most_deliverd_supercat	Category most delivered	Categorical
income_segment	Income category	Categorical
gender	Gender	Categorical
user_state_platform	User state platform	Categorical
days_on_platform	Number of days on the platform	Numerical
NR_experienced_last_365d	No response experienced in last 365 days	Numerical
revenues_next_6_months	Revenue in next 6 months	Numerical
request_id	Unique identifier	Key

Models Tried

Model Type	Model	Why?
Classical ML	Logistic Regression	→ Occam's razor: if simpler model can do it why complicate? → Gives us a baseline → Easy to interpret and explain → Parametric approach
Classical ML	XGBoost Classifier	→ Non-parametric and flexible approach → State-of-the-art for tabular data → Cross-validation supported out of the box → Robust towards NULL values → No normalisation required

Other models I might have tried if I had more data and time

Model Type	Model	Why?
Transformer	Tab-transformer https://huggingface.co/keras-io/tab_transformer	→ Pretrained for tabular data → Boasts performance as good as and sometimes better than XGBoost
	TabPFN https://huggingface.co/Prior-Labs/TabPFN-v2-clf	

Tech Stack

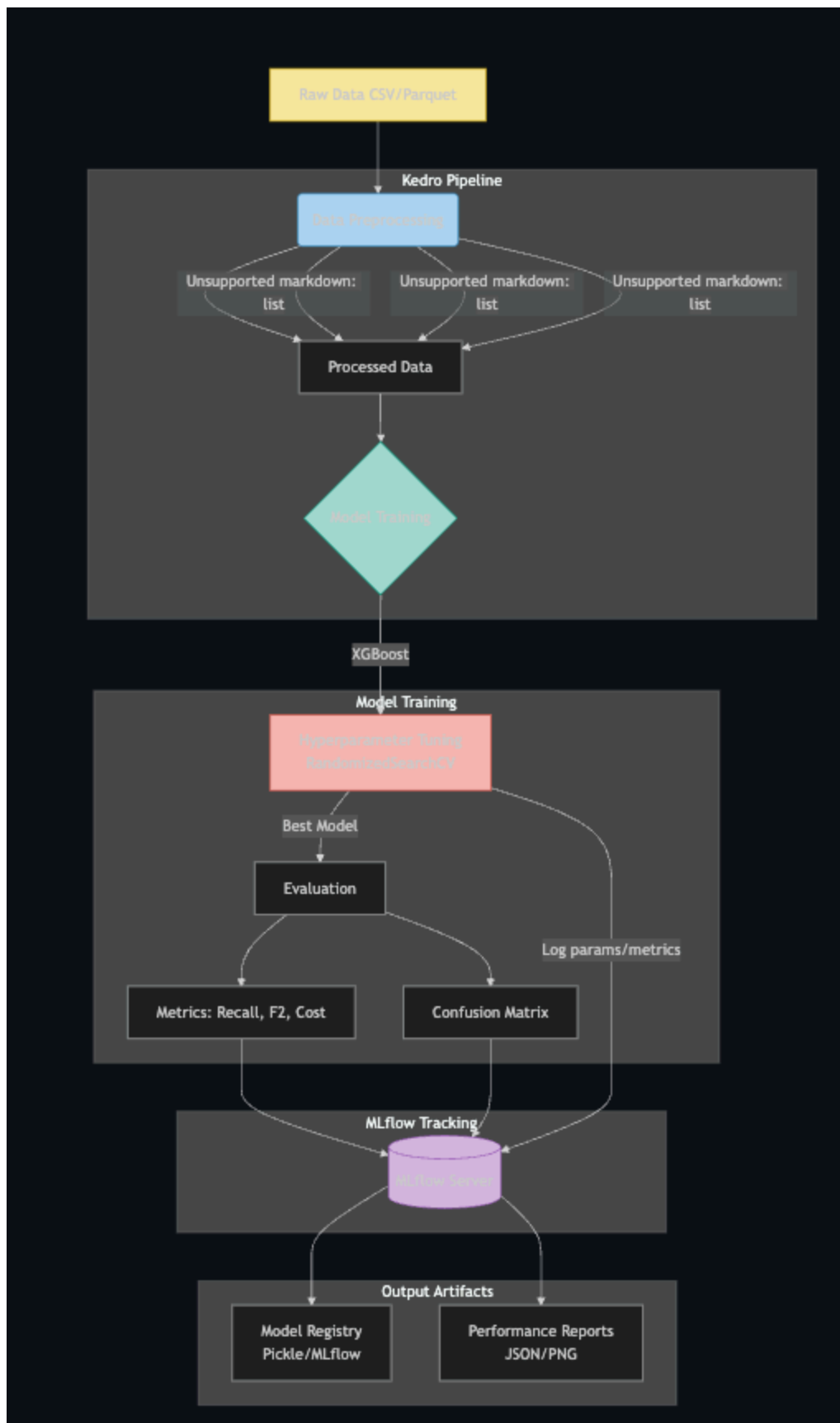
Element	Description
Python	→ No brainer, although most tasks could also be done in R
Conda	→ To maintain clear dependencies and to make the project reproducible
Kedro	→ Helps create pipeline for various tasks such as data processing, model training, evaluation, inference, etc. → Helps abstract data sources by maintaining clear catalog and maintains all constants in a parameters file → Helps version data and results along with storing model artefacts → Comes with out-of-box integration with docker, MLflow, Airflow, etc. for productionisation
MLflow	→ For experiment tracking, artefact versioning and tracking important metrics → Also, helps models can be registered and used as endpoints for production

Technical Overview

Please find the completely replicable code along with data artefacts and MLflow runs (in /mlruns) in the github repository here:

https://github.com/VishalSharma0309/uc_no_response/tree/develop

Note: There are better ways to share data and mlflow runs (through a shared deployed server) but given the current constraints and small size, seemed like a good idea.



Project Structure

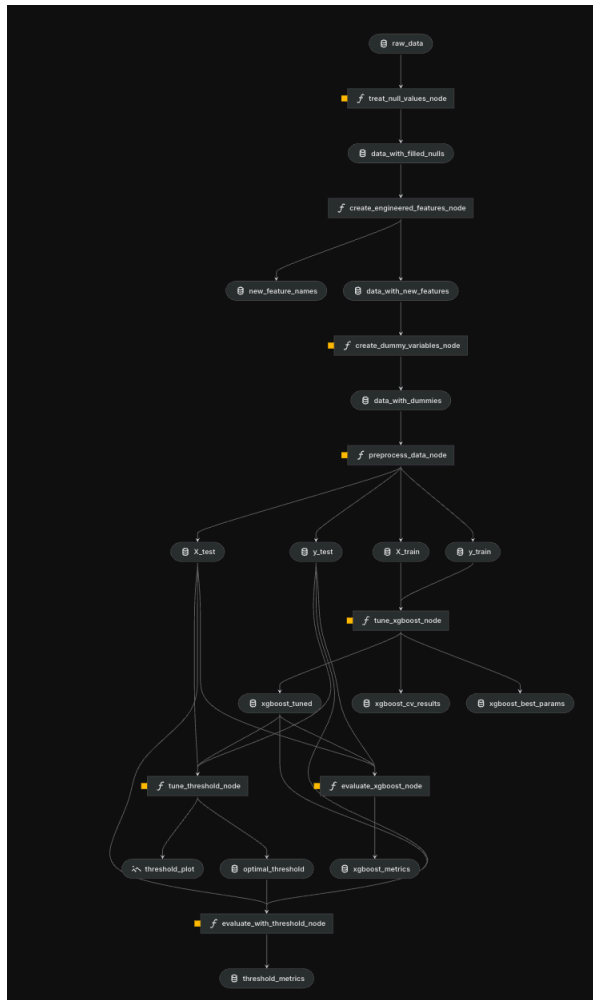
Data Catalog	The project uses a Kedro data catalog <u>conf/base/catalog.yml</u> to define datasets.	→ <u>Raw data</u> CSV stored in data/01_raw/ → <u>Intermediate data</u> Parquet files stored in data/02_intermediate for various stages of data processing → <u>Feature engineering data</u> Parquet files stored data/04_feature/ for feature engineering → <u>Models</u> Stored in data/06_models/ stores various versions of logistic regression and XGBoost model → <u>Reporting</u> Metrics and visualisations stored in data/08_reporting/
Parameters	The project has extensive parameters defined in <u>conf/base/parameters.yml</u>	→ Target column → Data preprocessing settings → Feature categorisation → Model parameters → Tuning grid for XGBoost → Cost matrix

Pipelines	Pipelines can be found in <code>src/uc_no_response/pipelines</code> where its further divided into <code>data_processing</code>, <code>modeling</code> and <code>evaluation</code>. Majorly I have created two pipelines: → Data Processing Pipeline → Modeling Pipeline (which also includes the evaluation pipeline) Registered pipelines can be seen in <code>src/uc_no_response/pipeline_registry.py</code>	→ Data Processing Pipeline contains various nodes such as <code>treat_null_values</code>, <code>create_engineered_features</code>, <code>create_dummy_variables</code>, <code>normalize_numerical_columns</code> (used only for LR) and <code>split_data</code> → Modeling pipeline includes training the model, hyperparameter tuning XGBoost using CV, threshold tuning for maximum cost and evaluation → Evaluation also contains feature important and SHAP summary
Notebooks	The project includes exploratory notebooks used for various evaluations and experiments	→ <code>01_eda.ipynb</code> Initial exploration of raw data → <code>02_data_cleaning.ipynb</code> Data cleaning operations → <code>03_eda_clean_data.ipynb</code> Further exploration on cleaned data → <code>04_feature_engineering.ipynb</code> Feature creation and analysis

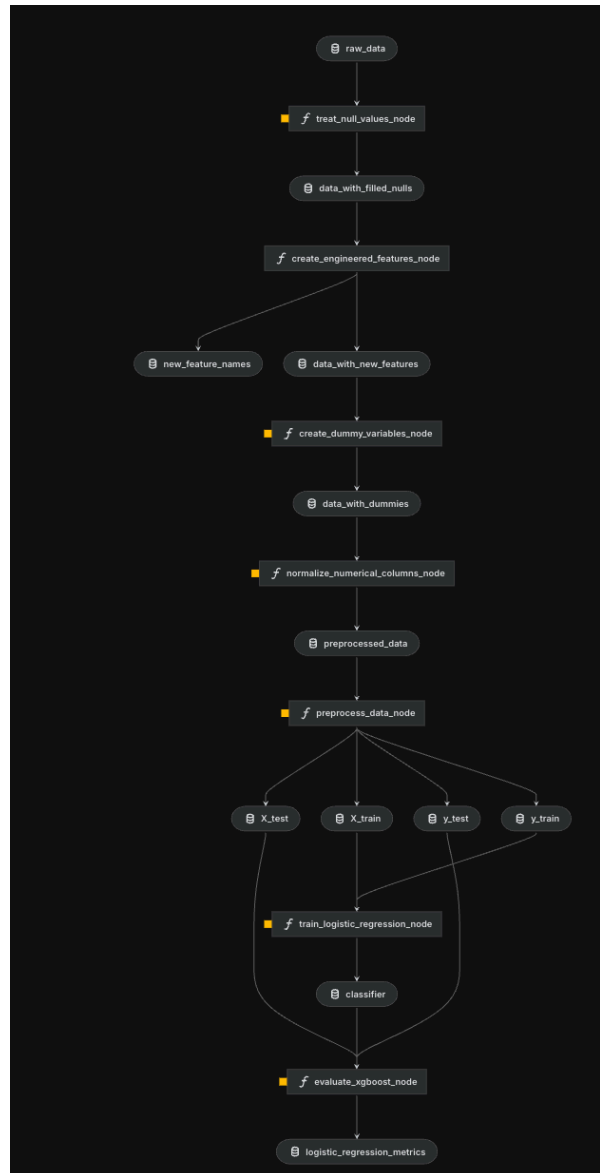


Kedro also has a bonus `kedro viz` functionality which let's us visualise the pipelines

XGBoost Tuning and Training



Logistic Regression Training

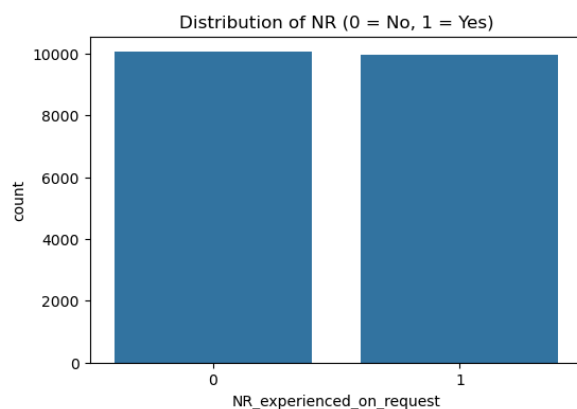


Key Features of the Project

- Cost-sensitive modeling: Uses a custom cost matrix to optimize model predictions
- Feature engineering: Creates derived features like delivery intensity and customer tenure
- Model tuning: XGBoost hyperparameter tuning with randomized search
- SHAP integration: For model explainability (mentioned in catalog)
- MLflow tracking: For experiment tracking and model versioning

EDA Insights

Data Imbalance



→ Data seems balanced across the two classes

- "Salon at Home" is the biggest "most delivered super-category"
- Although we have high NULL percentage for income segment but for the data available, most our customer base is 25L+ followed by 15-25L followed by 10-15L
- Most requests come from Male customers closely followed by Female
- Most our customers are "Engaged" which means repeated customers

NULL Handling

NULL percent for each predictor were as follows:

```
Unnamed: 0      0.000
NR_experienced_on_request  0.000
no_of_supercats_tried_last_30d  15.075
no_of_supercats_tried_last_90d  15.075
no_of_supercats_tried_last_365d  15.075
no_of_deliveries_last_30d  15.075
no_of_deliveries_last_90d  15.075
no_of_deliveries_last_365d  15.075
service_deliverd_aov_last_30d  58.910
service_deliverd_aov_last_90d  37.230
service_deliverd_aov_last_365d  20.735
most_deliverd_supercat  15.075
income_segment  75.295
gender  0.085
user_state_platform  15.075
days_on_platform  15.075
NR_experienced_last_365d  0.085
revenues_next_6_months  7.610
request_id  0.000
dtype: float64
```

After cleaning:

→ EDA reveals most NULL values were coexisting

Following steps were used to clean NULL values:

1. Remove rows with more than 80% null values
2. Convert NULL aov features with zero if corresponding deliveries is zero
3. Remove NULLS coexisting for number of super categories tried and service delivered
4. Fill remaining NULLS with median for numerical features and mode for categorical features

The above steps lead to zero NULLS for almost all features. Just two features income segment and revenues_next_6_months was left.

The later had just over 5% NULLS which I decided to fill with medians while I decided to not use

```

Unnamed: 0      0.000000
NR_experienced_on_request  0.000000
no_of_supercats_tried_last_30d  0.000000
no_of_supercats_tried_last_90d  0.000000
no_of_supercats_tried_last_365d  0.000000
no_of_deliveries_last_30d  0.000000
no_of_deliveries_last_90d  0.000000
no_of_deliveries_last_365d  0.000000
service_deliverd_aov_last_30d  0.000000
service_deliverd_aov_last_90d  0.000000
service_deliverd_aov_last_365d  0.000000
most_deliverd_supercat  0.000000
income_segment  73.217545
gender  0.000000
user_state_platform  0.000000
days_on_platform  0.000000
NR_experienced_last_365d  0.000000
revenues_next_6_months  5.398881
request_id  0.000000
dtype: float64

```

income_segment in modeling since it had too many NULLS



~15% rows were removed while cleaning NULL values

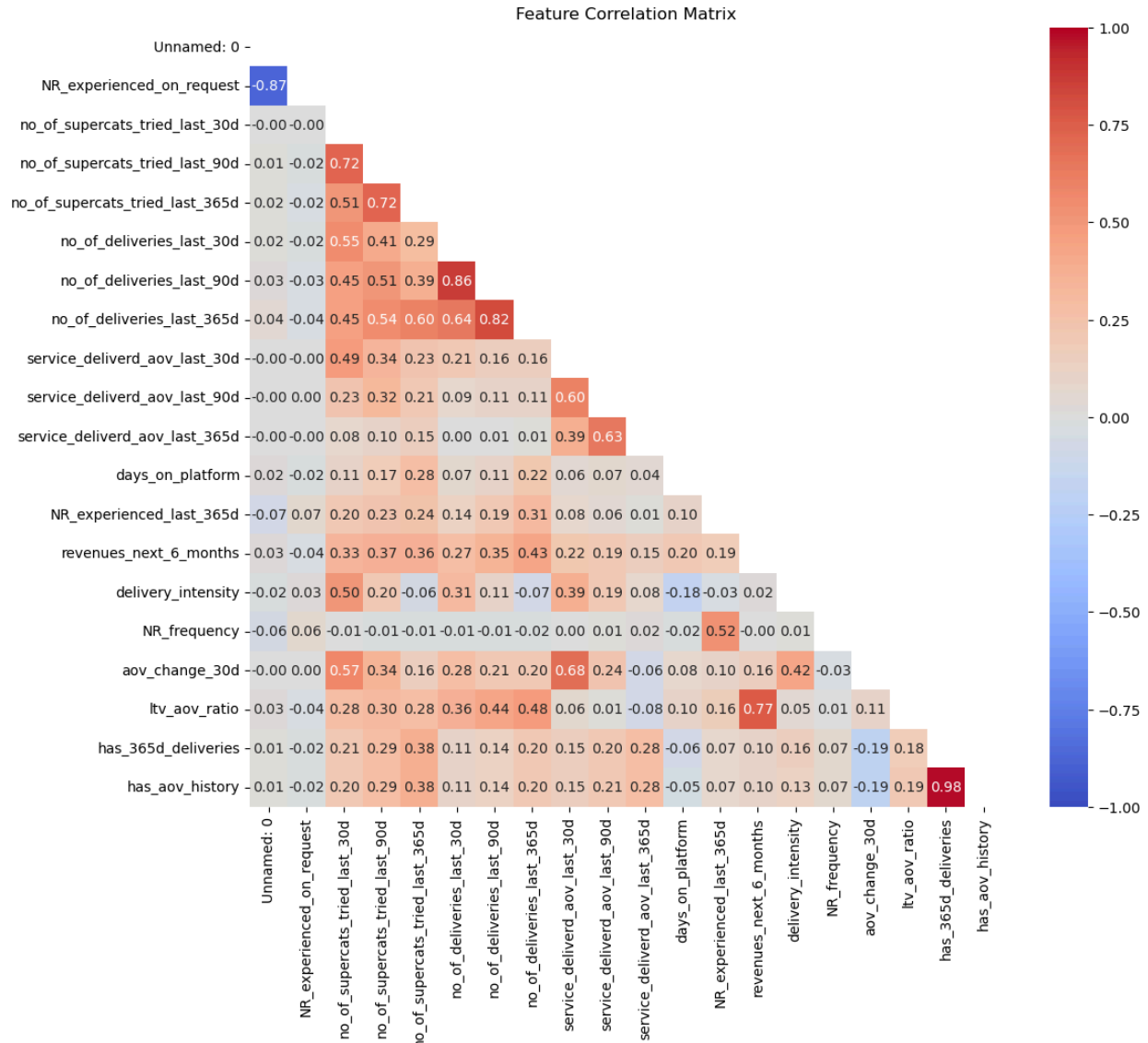
Feature Engineering

Feature	Description
delivery intensity	Number of deliveries (last 30D) / Number of deliveries (last 365D)
NR frequency	NR experienced (last 365D) / Number of deliveries (last 365D)
AOV change 30D	[AOV service delivered (last 30D) - AOV service delivered (last 365D)] / AOV service delivered (last 365D)
LTV AOV ratio	Revenue next 6 months / AOV service delivered (last 365D)
Customer Tenure Category	<500, [500, 1000], [1000, 1500], [1500, 2000], >2000 based on days on platform

	dtype	missing_pct	zero_pct	n_unique	mean	median	std	skew
delivery_intensity	float64	0.000000	0.516161	301	0.159459	0.000000	0.265185	2.121343
NR_frequency	float64	0.000000	0.848572	172	0.031019	0.000000	0.121531	7.772469
aov_change_30d	float64	0.000000	0.132470	7011	-0.442818	-0.541845	0.627148	1.528760
ltv_aov_ratio	float64	0.000000	0.071357	15322	6.865672	4.265205	9.935182	7.834451
customer_tenure_category	category	0.000000	0.000000	5	nan	nan	nan	nan

	Correlation with Target
NR_experienced_on_request	1.000000
NR_experienced_last_365d	0.069853
NR_frequency	0.063786
delivery_intensity	0.030923
aov_change_30d	0.004428
service_deliverd_aov_last_90d	0.003437
service_deliverd_aov_last_30d	-0.000429
no_of_supercats_tried_last_30d	-0.001743
service_deliverd_aov_last_365d	-0.002676
no_of_supercats_tried_last_90d	-0.015062
days_on_platform	-0.018693
no_of_deliveries_last_30d	-0.019607
has_365d_deliveries	-0.019944
has_aov_history	-0.020790
no_of_supercats_tried_last_365d	-0.024008
no_of_deliveries_last_90d	-0.032087
ltv_aov_ratio	-0.035703
revenues_next_6_months	-0.036833
no_of_deliveries_last_365d	-0.042500
Unnamed: 0	-0.868272

→ Correlation with the engineered features is on the higher side compared to original features



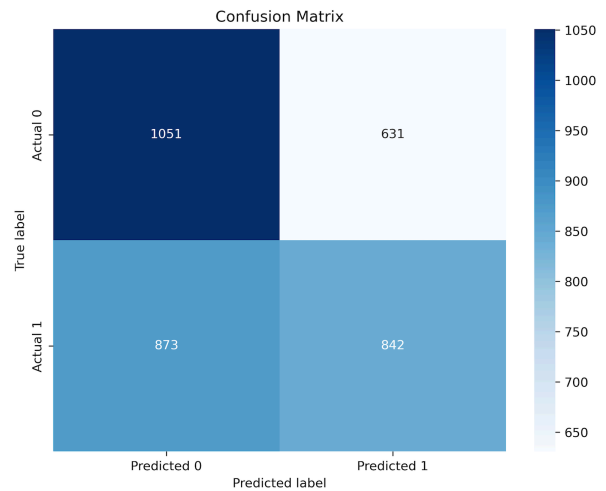
- None of the features show a high correlation with the response (which is expected)
- Number of super-categories tried and number of deliveries features have a strong positive correlation between themselves
- AOV values across timeframes show a strong correlation between themselves
- AOV values are also strong correlated with number of super-categories tried

Model Performance

1. XGBoost Classifier

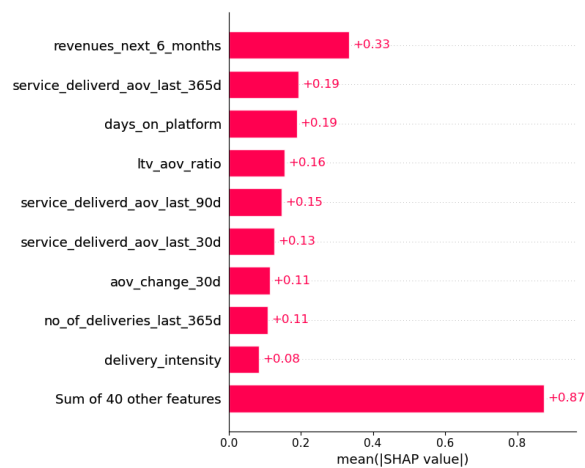
Metric	Score
Total Cost	-225900
Cost per instance	-66.50
f1	0.5282
Recall	0.4909
ROC AUC	0.5924
Precision	0.5716
Accuracy	0.5572

Confusion Matrix

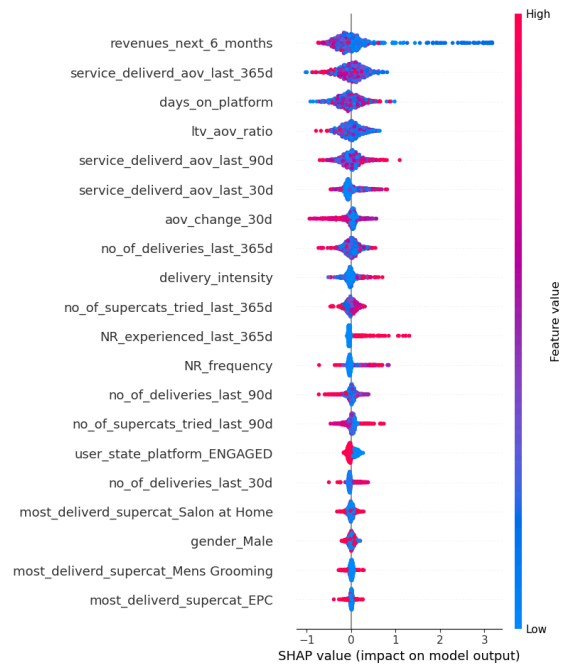


SHAP Analysis

SHAP Feature Importance



SHAP Summary



Revenues next 6 months is the most influential feature in the model along with service delivered AOV (365D) and days on platform. Engineered feature LTV AOV ration is also important.

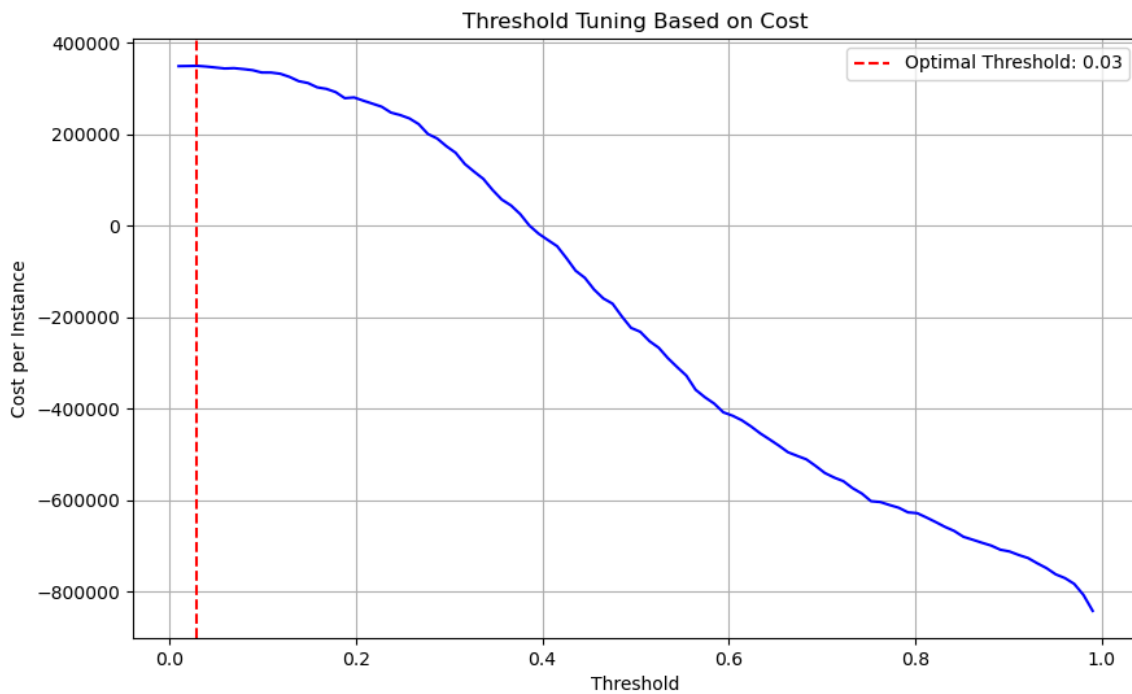
Hyperparameter Tuning

Parameter	Value
best_max_depth	9
best_learning_rate	0.13177599587339506
n_iter	50
best_min_child_weight	6
cv_folds	5
search_strategy	randomized
best_n_estimators	249
best_gamma	0.16360295318449863
best_colsample_bytree	0.7708431154505025
best_subsample	0.8136357677501768

Metric	Value
best_cv_score	0.5162148817117033

- max_depth of 9 suggests a deep tree with ability to understand complex patterns
- Gamma of 0.1636 suggests mild regularisation is in play, avoiding overfitting
- Learning rate is on the higher end, which means we need fewer trees
- Colsample_bytree suggests we use 77% of the available features, which avoids overfitting

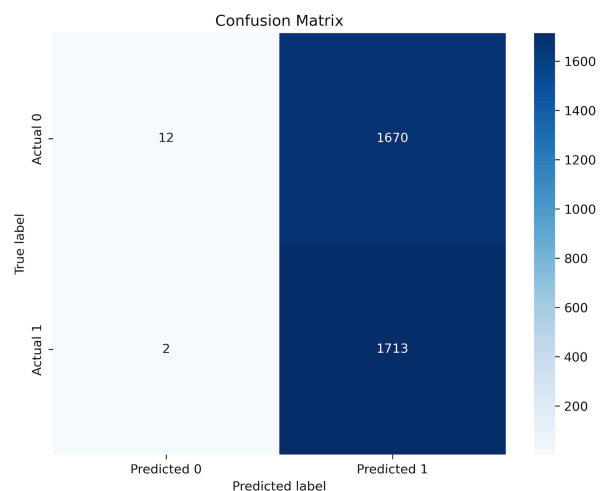
Threshold Tuning



→ Cost per instance shows a monotonically decreasing trend with threshold. To maximise the cost, we choose a very small value of threshold which is **0.02979**

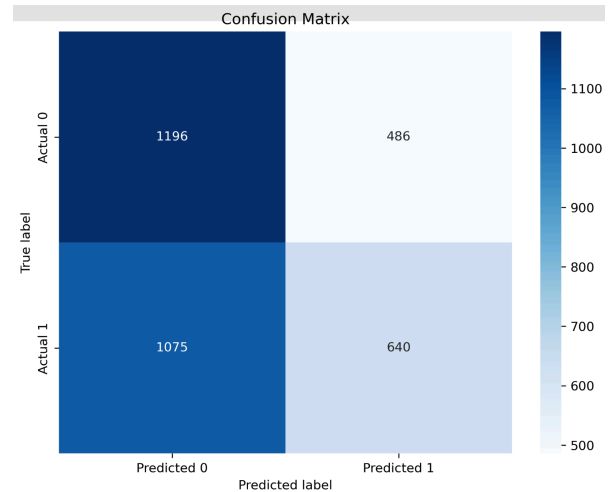
Tuned Threshold Metrics

Tuned Metric	Score
Total Cost	350200
Cost per instance	103
f1	0.6720
Recall	0.9988
ROC AUC	0.5924
Precision	0.5063
Accuracy	0.5078

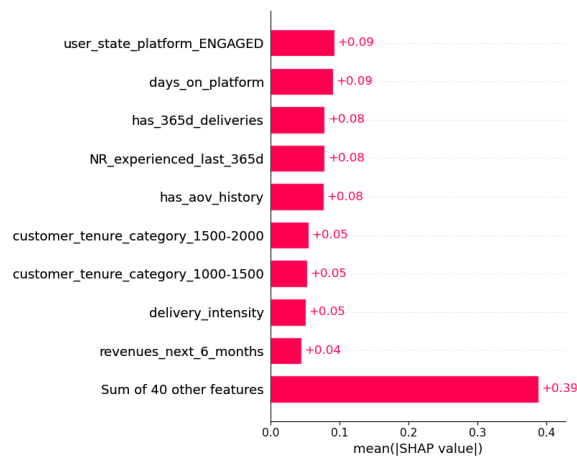


2. Logistic Regression

Metric	Score
Total Cost	-378700
Cost per instance	-111.48
f1	0.4505
Recall	0.3731
ROC AUC	0.5663
Precision	0.5683
Accuracy	0.5404



SHAP Analysis

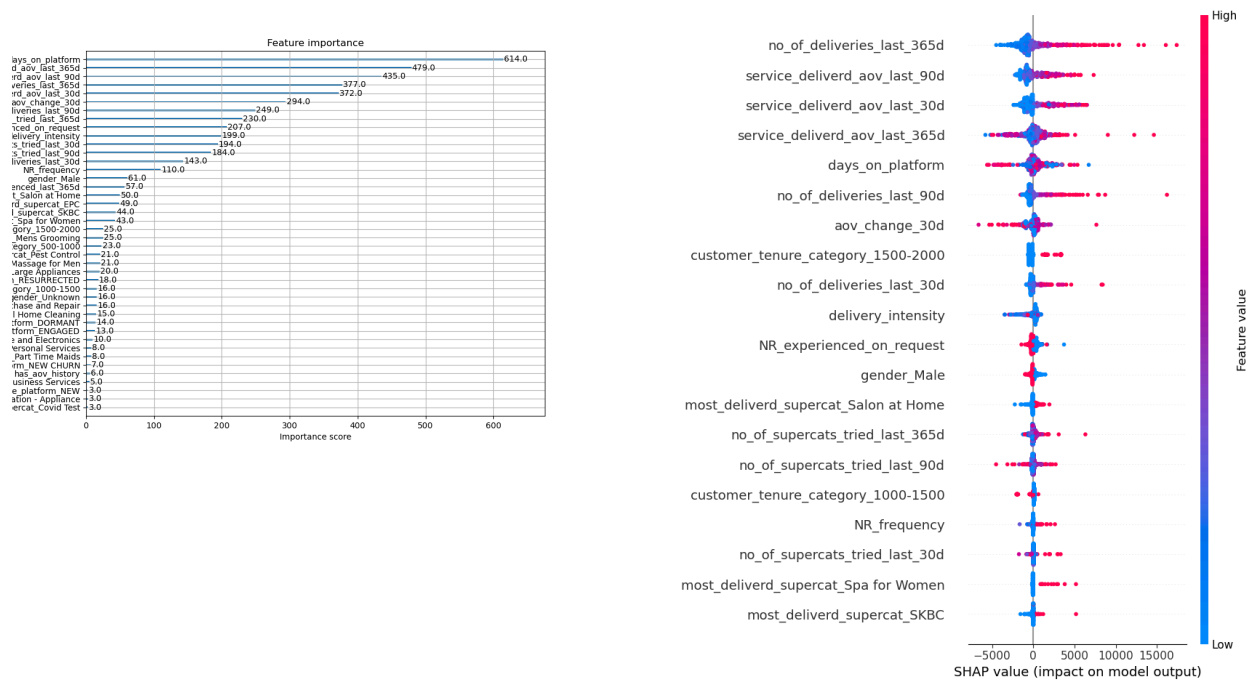


[illegible]

→ I assumed here that **revenues_next_6_months** is a proxy for LTV

- Model: `revenues_next_6_months ~ NR_experienced_on_request + controls`.

20



→ We can notice here that although not the most important but NR_experienced_on_request is an important feature while trying to predict LTV.

→ Hence a value of 1 for NR exp on request can negatively impact the LTV